Introduction

The motivating question behind this project was to determine if dynamic systematic investment plans are capable of outperforming a fixed systematic investment plan based on 20 years of S&P 500 data.

The motivation behind the project was to apply multiple types of data structures to simulate 20 years of investing. This required use of linked lists, binary search trees, circular queues, stacks and queues to store, evaluate and simulate the data. Considering that ESE224 is a data structures class, this project was a good opportunity to apply the content of the class to a real world problem and create a useful tool. Additionally, ESE224 is a C++ class, so by writing the entirety of the project in C++ it also reinforced important C++ topics that were covered such as inheritance, object oriented programming, templating and encapsulation.

The data used came from SPX, AMZN and NVDA. SPX is the S&P 500 index, which is used to represent the average state of the US economy. AMZN is the stock of Amazon Incorporated, the popular e-commerce and technology company. NVDA is NVIDIA Corporation. This particular stock was included due to its high volatility to see how the strategies respond to drastic changes. There was data for each day of the stock's or index's existence and for each day there were various values. One value was the value of the stock or index at the time at which the market opened. Another value was the value of the stock or index at the time at which the market closed. Another value was the highest value that the stock or index hit during the day. Another value was the lowest value that the stock or index hit during the day. Another value was the adjusted close, which is an altered closing price that incorporates context behind what financial moves companies made that day (such as dividends or stock splits) that would have altered stock prices. The last value included was volume, which represents the total number of shares sold (or bought) during that day. Obviously the data excluded weekends and holidays (and any other times at which the markets were closed). These datasets were very large, especially the S&P 500 for which the data starts on December 30, 1927. This is much larger than the Nvidia data which started on January 1, 1999 or the Amazon data, which started

on May 15, 1997. As previously stated, this project only evaluated 20 years of data, so the only data used by the project is anything between January 1, 2000 and January 1, 2020.

Data Structure Design

1. Linked List

The linked list was a doubly linked list under class PriceHistory used to hold data for a particular stock or index.

The doubly linked list consisted of priceNodes structs that held all the relevant data fields for a trading day (date, open, high, low, close, volume) as well as pointers to previous date and next date. Obviously the class PriceHistory had a constructor that created an empty doubly linked list and a destructor that cleaned up all data associated with the list. The constructor method is O(1) as it has no loops and the destructor is O(n) as it has a while loop to traverse. Then there is an append function that adds a node to the end of the list, which is necessary for filling out the data structure with the appropriate data. If the list is empty, it adds a node to the list, if the list is not empty it adds a node to the back of the list. Because there is a tail pointer, no list traversal is needed and time complexity is O(1). Next is the findByDate method which returns a pointer to the PriceNode at a current date. This is useful for trading strategies that require data from a certain date as the PriceNode will have that data publicly available. Since this required traversal of the list a loop was necessary so time complexity is O(n). The next method is printByRange which takes a starting and ending range and prints formatted data for this time period. This also required traversal and so it is also O(n). Lastly there were two iterator classes, an iterator that would start at a node and work forward and a reverseIterator that would start at a node and work backward. The iterator operations as advancing to the next node (operator++), accessing the current node (operator*), and comparing two iterators (operator!=), each perform a single pointer operation, so all are O(1).

This data structure was necessary for price history because price history needs to be able to be viewed chronologically and this requires traversal and a stack or queue wouldn't allow for that. Additionally time is linear, and so a non-linear data structure like a binary search tree also wouldn't fit the requirements.

2. Binary Search Tree

StockBST is a binary search tree that uses the sorting ability of binary search trees that organize stocks by annual return percentage.

Class StockBST uses a struct BSTNode that stores the data associated with each stock as well as pointers to left and right children of the node. Then StockBST has a constructor which creates an empty tree, but making the root null. No loops in this method so time complexity O(1). The destructor uses a helper function clear which calls clearHelper on the root, clearHelper is a second helper function that starts from a given node (root in this case) and recursive deletes all left and right children of that node before deleting the node itself. Time complexity due to recursive calls is O(n). Next is the insert method which adds a new entry or if a new entry is a duplicate it adds to the right subtree. This uses an insertHelper which takes an initial node and data associated with the node to be added. Then it follows the recursive traversal algorithm of checking the new node key against the current node key and going left if the new node key is less than current node key or right otherwise. Time complexity due to recursion is best case O(log n) or worst case O(n). Next is the search function. It uses searchHelper starting at root with a given key. searchHelper recursively checks each node to try to find the key and returns a pointer to the node if found or null pointer if node isn't found. Time complexity due to recursion is again best case O(log n) or worst case O(n). The next method is the rangeSearch function which finds all stocks in between a given low and high key and places them into a vector. Then it uses a recursive rangeSearchHelper to find if the current key is between the given range. If it is in a given range it adds to the vector. This time complexity, due to recursion, is best case O(log n + k) or worst case O(n) with time k being the number of nodes with keys in the range given. Then there is inOrder, preOrder, postOrder which use recursive

helpers to follow the standard procedure for ordering in each given way. Each order function has time complexity O(n) due to complete traversal of the tree. Then there is a height function that uses heightHelper to start from root and go all the way down (along the longest path) to external nodes. It then returns the length of this path as an integer. This has time complexity O(n) due to complete traversal of the list. Lastly there is findMax which continuously goes right until it cannot anymore, it then returns a pointer to the node with highest key value. Time complexity of worst case O(n) or best case O(log n).

A binary search tree was used due to its ability to create sorted data, a queue or stack would not be able to do this. Additionally a linked list would be able to do this but it would require complex insertion between two nodes that makes it a much less desirable option than the binary search tree.

3. Circular Queue

The circular queue was used mainly to compute a rolling average of trading day close prices.

The constructor for the circular queue required a capacity argument, which would become the max size of the queue. So to compute a 50 day average of a stock a queue of capacity 50 was needed. Then the constructor created an array of size capacity. The destructor simply cleaned up that array. Both constructor and destructor were O(1). Next there was an enqueue method. The class had a head and tail that maintained the oldest and newest values in the queue, respectively. The enqueue function always inserted at tail and used tail = (tail + 1) % capacity; to ensure the queue would be overwritten when full. If the queue was full the enqueue overwrote the head and then updated head and tail. No loop in enqueue, so O(1). Next, dequeue would return the value stored at head and advance head to indicate the dequeue value could be overwritten. Again, no loops so O(1). Considering the point of the circular queue was to compute a rolling average, there needed to be a getAverage function. This simply started at head and computed the sum of all elements between head and tail which is the relevant data in the queue. Then it divided that sum by the number of values it summed and returned that average.

Due to the looping through the values, the time complexity is O(n). Additionally, there is a peek method that returns the value at head without removing it, and isFull, isEmpty, and getCount accessors that each read a single member variable. All four are O(1) as they involve no loops.

This data structure was used to compute a rolling average. So the capability of a circular queue to overwrite the old values was used to only keep the most recent data. If this was done with a different structure it would require a removal of the oldest value, all other values to be shifted and then an insertion of the most recent value. This is more complicated than simply overwriting the oldest value.

4. Stack

The stack was used to undo trades, and track the history of trades made. This was useful when simulating and tracking what moves were made.

TradeStack consisted of a struct TradeRecord that has data about each trade. Then TradeStack had struct StackNode which each contained a TradeRecord and a StackNode pointer. These pointers always point to the node below them on the stack. The constructor for TradeStack created an empty stack with time complexity O(1). The destructor just deletes all StackNodes with a while loop, so time complexity O(n). After this there is push, which simply adds a node above the top and updates the top. No loop so time complexity O(1). There is also a peek method that returns the top TradeRecord without removing it, and isEmpty and getSize accessors that each read a single member variable, all O(1). Pop method extracts TradeRecord from the top StackNode moves the top pointer down one node, deletes the popped node and returns the trade data. If the stack is empty it returns the default reversion of the TradeRecord. No loop so time complexity O(1). The last important function is a printAll that starts at the top of the stack and prints all trades made from most recent to least recent. This uses a loop to traverse, so it is time complexity O(n).

A stack was necessary here because of the ability to pop off the most recent trade, which is crucial to implement undo. This is not possible in a queue because the top of the queue is the

oldest trade not the most recent trade. A linked list could be used for this purpose, but a stack is simpler. Additionally, there is no need for a binary search tree because it would be too complex for what is needed.

5. Queue

Queue is used to create a list of orders where top order is executed after a prompt from Portfolio class.

Similarly to the TradeStack, OrderQueue uses an Order struct to hold details of the order. Then the OrderQueue has QueueNode structs that have an Order and a QueueNode pointer to maintain the structure of the queue. The OrderQueue methods include a constructor which makes an empty queue with time complexity O(1). There is also a destructor that traverses the list and deletes each QueueNode, this traversal means a loop, so it is time complexity O(n). Next there is method enqueue, this takes an Order and adds it to the back of the queue. The OrderQueue has a front and back pointer to help with this function as well as dequeue. If the queue is empty, the front and back are the new Order. No loop for this one so time complexity is O(1). Dequeque then returns and removes Order at front of queue, if queue was empty it returns default of Order. No loop, so also time complexity O(1). There is also a peek method that returns the front Order without removing it, and isEmpty and getSize accessors that each read a single member variable, all O(1). There is also a printAll method that starts from the front of the queue (oldest orders) and prints all newer orders, this also uses a loop to traverse the queue, so complexity is O(n).

Since these orders are made following First-In-First-Out principle, then it makes sense to use a queue, which also follows First-In-First-Out. A stack doesn't follow First-In-First-Out so it wouldn't work and a non-linear structure like a binary search tree wouldn't work. A doubly linked list could work, but it is more memory efficient to just use a queue.

[AI Usage — Section 2: Data Structure Design]

Prompt: "void printRange(const string& startDate, const string& endDate) const; what is this supposed to print?" AI Output: Explained what each node should print during a date-range traversal. Modification: Wrote my own implementation using the conceptual explanation.

Prompt: "what about using CSVParser::dateinrange?" AI Output: Warned this would create a circular include dependency between PriceHistory and CSVParser. Modification: Kept date comparison inline using direct >= and <= to avoid the circular include.

Prompt: "Iterator::Iterator(PriceNode* node) { } [asking how to fill it in]" AI Output: Showed initializer list syntax for storing the node pointer. Modification: Used the simpler cur = node; assignment instead for readability.

Prompt: "Iterator& PriceHistory::Iterator::operator++() { } [asking how to fill it in]" AI Output: Explained pre-increment advancing the internal pointer forward. Modification: Wrote my own version and fixed a variable name mismatch (cur vs current) by checking my header.

Prompt: "[VS Code error] declaration is incompatible with PriceHistory::Iterator" AI Output: Explained that the return type needs full scope PriceHistory::Iterator& when defined outside the class body. Modification: Applied full scope prefix to all iterator method definitions in the .cpp.

Prompt: "PriceHistory::PriceNode& PriceHistory::Iterator::operator*() { return current; } [asking if correct]" AI Output: Pointed out I needed to dereference with *current, not return the pointer directly. Modification: Fixed dereference and removed unnecessary PriceHistory:: prefix on PriceNode.

Prompt: "bool PriceHistory::Iterator::operator!=(const Iterator& other) const { } what should this do" AI Output: Explained it should compare the underlying node pointers, not the iterator objects. Modification: Wrote return current != other.current.

Prompt: "if other is an iterator why use .current? / why not just compare the iterator if they are pointers / what would comparing objects do / im confused why do any point to nullptr" AI Output: Clarified how iterators wrap pointers and how nullptr serves as the end sentinel for begin/end. Modification: Used this to correctly implement begin/end and rbegin/rend.

Prompt: "[full PriceHistory.cpp pasted for review]" AI Output: Caught that rbegin() and rend() were swapped. Modification: Fixed rbegin to return ReverseIterator(tail) and rend to return nullptr.

Prompt: "What is a getter and a setter" AI Output: Explained encapsulation patterns for accessor methods. Modification: Used the explanation to write my own getters and setters across all classes.

Prompt: "return TradeRecord(); // Return default record if stack is empty / really it does" AI Output: Discussed risks of silently returning default values on empty pop. Modification: Kept default return with a cout error message since rubric says preconditions are caller responsibility.

Prompt: "[showed destructor calling pop() in a loop]" AI Output: Suggested directly walking and deleting nodes to avoid unnecessary TradeRecord copies. Modification: Rewrote destructor with manual pointer walking.

Prompt: "void OrderQueue::printAll() const { ... while (current != back) { ... } }" AI Output: Caught that current != back skips the last node in the queue. Modification: Changed condition to current != nullptr so all nodes are printed.

Prompt: "why current != nullptr? / next is auto initialized to nullptr?" AI Output: Confirmed next is not auto-initialized and must be set explicitly on node creation. Modification: Made sure all QueueNode constructors explicitly set next to nullptr.

Prompt: "yea but what if it is called in etf / yea but if a stock is called there now wont be a bottom line" AI Output: Discussed formatting border line behavior when one printSummary calls another. Modification: Decided to omit the === border lines from ETF::printSummary entirely.

Prompt: "double calculate10YearCAGR() const; [asking for help]" AI Output: Gave a version using head and tail prices. Modification: Recognized it needed the last 10 years specifically, so used a reverse iterator to walk back from tail to find the correct start price.

Prompt: "void ETF::setExpenseRatio(double ratio) const { expenseRatio = ratio; } [showed code]" AI Output: Pointed out setters cannot be const since they modify member state. Modification: Removed const from the setter signature.

Prompt: "why isnt extract year working [showed undefined identifier error]" AI Output: Identified missing CSVParser:: scope prefix and missing include. Modification: Added #include "CSVParser.h" and added the scope prefix to the call.

Prompt: "what is . vs ->" AI Output: Explained the rule: object uses dot, pointer uses arrow. Modification: Applied this rule consistently throughout all class implementations.

Prompt: "[partial loadHistory implementation showing find + erase pattern]" AI Output: Showed how to continue the same pattern for remaining CSV fields. Modification: Applied the same find + erase approach to High, Low, Close, Adj Close, and Volume.

Prompt: "what is going on though / so you not moving the cursor? / what is stod / why pos+1? / and erase erases from 0 to pos-1?" AI Output: Clarified how find + erase works on strings and what stod does. Modification: Fixed an off-by-one error in the Low field erase that was missing the +1.

Prompt: "what is ... in catch / why cant i do catch()" AI Output: Explained that catch(...) catches all exceptions and empty parens are invalid syntax. Modification: Wrapped CSV row parsing in try/catch(...) to silently skip malformed rows.

Prompt: "[showed loadHistory with static keyword in cpp]" AI Output: Pointed out static belongs only in the header declaration, not the .cpp definition. Modification: Removed static from all CSVParser method definitions in the .cpp file.

Prompt: "[showed cpp with volume reading still using find]" AI Output: Noted volume is the last field with no trailing comma so stol(line) works directly. Modification: Replaced find + erase for volume with a direct stol(line) call.

Prompt: "why does stacknode point to top?" AI Output: Clarified how new nodes link to the previous top so the pointer chain is not lost. Modification: No change needed; confirmed my existing logic was correct.

Prompt: "[showed destructor with !isEmpty without parens]" AI Output: Noted isEmpty needs parentheses to actually call the function. Modification: Fixed to !isEmpty().

Prompt: "why isnt private data handled in constructor?" AI Output: Confirmed vector<T*> auto-initializes to empty so no constructor body is needed. Modification: Left constructor body empty as confirmed.

Trading Strategies

1. Fixed SIP Strategy

The fixed SIP strategy is the baseline strategy and because of this it is fairly simple. It goes through the price history of the stock or index the strategy is being used on. It takes a start year and end year for the evaluation of the price history and only works if data is within the appropriate range. If it is within the appropriate range then it checks if it is the first trading day of a new month, invests monthly capital, an argument of the function, into the stock. It computes shares bought by simply dividing monthly capital by the close price of the stock that day. After looping through the date range of the data with this investment strategy it calculates some overall final values of the strategy. This strategy is expected to closely mirror the growth of the stock and index which it is testing. It is blindly investing in the stock regardless of any other consideration, so it would make sense if the strategy's growth rate was close to the stock's growth rate within the same time period.

2. Dynamic SIP Strategy

The Dynamic SIP Strategy combines elements of the Golden Cross Strategy and a tiered buying strategy, and does so using eight parameters total, which had to be simulated upon to find the best weights for each. Starting with the moving average windows, there are the shortWindow and longWindow parameters, which store the number of days to look back and keep moving averages of. The functionality of these parameters is very similar to that of the Golden Cross Strategy: if the shortWindow day average transitions to be greater than the longWindow day average, the strategy invests all available cash into shares of the stock. If the reverse happens, all stock shares are sold. The parameters mildDipThreshold and severeDipThreshold specify

percentages from a high within a specified amount of days in the past (fixed as lookbackDays) that the market has dropped to the current day. These values are used to create moderateDipThreshold, defined as the average between the two. These three parameters act as thresholds to buy stock. If the severe threshold is met, all cash is deployed regardless of the market status. If the moderate is met, only 2 times the mildMultiplier amount is invested (mildMultiplier being defined as a fraction of monthlyCapital to invest). If the mild threshold is met, only the mildMultiplier amount is invested. The tiered strategy always takes priority of the highest applicable threshold, so if a severe threshold is met, it will be the action taken that day. Finally, in the case of a rally, a percent gain parameter was added called rallyThreshold, which limits buying to the product of monthlyCapital times rallyMultiplier. Overall, the goal of this strategy was to give this simple trading algorithm as many weights as possible, such that simulation could unlock the best parameter values amongst the eight parameters. When backtested on the S&P 500 from 2000 to 2020, the optimal parameters were found to be 24, 234, 2.75, 18.75, 0.1, 62, 9.2, and 0.40 for shortWindow, longWindow, mildDipThreshold, severeDipThreshold, mildMultiplier, lookbackDays, rallyThreshold, and rallyMultiplier respectively. If tested on a different period of time, these parameters would need to be adjusted through simulation to find the optimal strategy over that time.

3. Golden Cross Strategy

The strategy uses two averages of recent stock performance to determine when to buy or sell. These averages are computed with the previously defined circular queue class. One average is over a short window and one average is over a long window. These windows can be adjusted, but for the simulation they are sent to 50 for short window and 200 for long window.

The logic of the strategy again loops through each day of data within a given year range. The strategy is also given a balance of monthly cash to work with, but it doesn't automatically invest each at the start of each month. It uses the 50 day average and 200 day average to try to find more advantageous times to invest. The specific way it does this is by checking if the 50 day average crosses over and exceeds the 200 day average, if this is the case the strategy invests

all available cash into shares of the stock. Now if the reverse happens, meaning the 200 day average crosses over and exceeds the 50 day average, all shares held of the stock are sold. Similarly to fixed SIP, totals are saved with each iteration to be presented as final values at the end of simulation. This would likely outperform the fixed SIP because at any indication of a decreased value of the stock, the shares are sold and at any indication of increasing value of the stock, more shares are bought. This strategy aims to act on trends recognized to maximize return.

4. Momentum Strategy

The momentum strategy uses past data of the stock to determine whether to sell or buy. Just like previous strategies, the strategy loops through all data and only acts when in between some start and end year. Additionally, it also adds monthly capital to account at the start of each trading month. Then at the start of each month it starts at the current day and evaluates the price some X days ago. Using the current price and this previous price it computes the trailing return, which is the return as a percentage from X days ago to now. For this strategy the number of days that the trailing return calculates from is the parameter. For the simulation, 126 is used (because it is about 6 months). The second parameter is momentum threshold, the strategy compares the trailing return to this momentum threshold to determine what action to take. During simulation 5% (or 5.0) is used for the momentum threshold. If the trailing return is greater than the momentum threshold, all available cash is invested in stock. If the trailing return is negative (meaning the stock lost money in the last X days), then the strategy sells all stocks that it does hold. If the trailing return is positive, but not greater than the momentum threshold given as a parameter, nothing is done. It only calculates this trailing return on the first trading day of each month and then it also only takes action on that first trading day of each month. Again it tracks all transactions in order to return totals associated with the strategy. This strategy also would likely outperform the fixed SIP strategy because it evaluates trends in the price to try to limit losses or maximize gains. However, for a very volatile stock a 6 month average might be too long to sufficiently capture a trend which would limit the effectiveness of this strategy.

[AI Usage — Section 3: Trading Strategies]

Prompt: "what is backtest? / so we simulate only one asset?" AI Output: Explained that backtest simulates one asset over a year range and returns a SimResult. Modification: Used this to understand the interface before implementing all four strategy classes.

Prompt: "the only question i have is why is lastClose = node.close after the if statement / why is it even in the for loop then / why doesnt node stay why cant it be accessed?" AI Output: Clarified that node goes out of scope after each loop iteration so lastClose must be saved. Modification: No change; confirmed my existing placement was already correct.

Prompt: "[showed early FixedSIPStrategy backtest with several bugs]" AI Output: Listed five bugs including a typo, missing lastClose update, calculations inside loop, and missing return. Modification: Fixed all five bugs.

Prompt: "yea but the constructor doesnt use m or d / so this compare by ticker tells the ai the means of what to swap" AI Output: Explained parameter name shadowing and why this-> is needed when names match member variables. Modification: Used short parameter names to avoid the shadowing issue entirely.

Prompt: "[showed nested for loop trying to construct ReverseIterator from Iterator]" AI Output: Showed that ReverseIterator must be constructed from a raw node pointer using &(*it). Modification: Changed construction to use &(*it) since my Iterator class has no ptr() accessor.

Prompt: "where does cash come from? / it isnt portfolio or anything / so you invest everything?" AI Output: Clarified that cash is a local simulation variable not connected to the Portfolio class. Modification: Confirmed buy logic invests both accumulated cash and the current month's contribution.

Prompt: "For this if trailingReturn > momentumThreshold but it is a new month does monthly capital get added to cash?" AI Output: Caught that monthly capital was only being added inside the buy branch. Modification: Moved cash += monthlyCapital and totalInvested += monthlyCapital to run unconditionally on first trading day of each month.

Prompt: "[showed full GoldenCrossStrategy.cpp]" AI Output: Identified four bugs: duplicate push_back lines, prevValues never set to true, MA queues not fed during warmup, and unconditional monthly investment. Modification: Made all four fixes.
Prompt: "ok rewrite the file using elitestrategy but rename it to dynamicsipstrategy / but it sells everything though? / why three layers? / what are the dip thresholds calculated as?" AI Output: Provided a three-layer hybrid strategy combining MA crossover, tiered dip-buying, and base SIP logic. Modification: Used as a starting point then modified logic and parameters to match my design goals.
Prompt: "[showed parameter sweep output where every combination produced exactly $626,781]" AI Output: Identified that cash was deploying every month regardless of dip signal so thresholds were never reached. Modification: Removed the unconditional monthly invest branch so cash only deploys on actual dip signals. Sweep showed real variation after the fix.

Results and Analysis

To evaluate the four implemented trading strategies, each was backtested on historical price data spanning January 2000 through December 2020, a twenty-year window deliberately chosen to capture two major market crashes (the dot-com collapse of 2000–2002 and the 2008 financial crisis), a prolonged bull market (2009–2019), and the sharp COVID-19 correction of early 2020. Each simulation assumed a fixed monthly contribution of $500, totaling $120,000 invested over the full period. All strategies were tested first on the S&P 500 index (SPX) as the benchmark, and then on Amazon (AMZN) as a representative high-volatility individual stock. Performance was evaluated across four metrics: final portfolio value, total return percentage, compound annual growth rate (CAGR), and maximum drawdown.

<u>SPX Results (2000–2020)</u>

Strategy	Invested	Final Value	Return %	CAGR %	Max Drawdown %	Trades

Fixed SIP	$120,000.00	$276,886.13	130.74%	4.27%	48.72%	240
Dynamic SIP	$120,000.00	$401,601.47	234.67%	6.23%	32.66%	461
Golden Cross	$120,000.00	$251,898.38	109.92%	3.78%	16.05%	18
Momentum Strategy	$120,000.00	$233,042.46	94.20%	3.37%	19.21%	120

From these results, it can be seen that Dynamic SIP produced the highest final value at $401,601.47, representing a 234.67% total return and a CAGR of 6.23%. This strategy outperformed Fixed SIP by over $124,000 in absolute terms and nearly two full percentage points in annualized return. Dynamic SIP also reduced maximum drawdown from 48.72% to 32.66%, a reduction of over 16 percentage points. This combination of higher returns and lower peak losses is the main conclusion from this section. The Dynamic strategy was structurally a better fit on both dimensions relative to the Fixed baseline.

The Golden Cross Strategy produced the lowest maximum drawdown of any strategy at 16.05%, but significantly underperformed on final value and CAGR. This is a direct consequence of its binary nature, with only 18 total trades over 20 years, the strategy sat in cash for extended periods during which monthly contributions accumulated uninvested. The cost of missing bull market exposure outweighed the benefit of avoiding drawdowns over this time horizon. The Momentum Strategy performed similarly, finishing with the lowest final value at $233,042.46 and a CAGR of 3.37%. Both market-timing strategies underperformed Fixed SIP in final value on SPX, demonstrating that exiting the market too conservatively over a long-term upward trend carries a significant opportunity cost.

<u>AMZN Results (2000–2020)</u>

Strategy	Invested	Final Value	Return %	CAGR %	Max Drawdown %	Trades

Fixed SIP	$120,000.00	$4,026,430.27	3255.36%	19.20%	63.62%	240
Dynamic SIP	$120,000.00	$4,591,075.39	3725.90%	19.99%	62.98%	890
Golden Cross	$120,000.00	$902,881.16	652.40%	10.62%	46.69%	20
Momentum Strategy	$120,000.00	$1,294,960.46	979.13%	12.63%	40.15%	153

The AMZN results show a much different performance landscape, and the four strategies produced different results. All four strategies produced significantly higher absolute returns than their SPX counterparts, reflecting Amazon's extraordinary price appreciation over the two-decade period. The drawdown advantage also vanished, with 62.98% for Dynamic SIP versus 63.62% for Fixed SIP, a difference of less than one percentage point. This convergence is explained by the nature of AMZN's price history as an asset with sustained, multi-year nearly parabolic growth which provided little opportunity for the death cross mechanism to add value, since exiting the position during a bearish signal means missing large subsequent gains. The dip-buying tiers similarly contribute less when dips are short-lived and recoveries are steep. It is also important to note that the Dynamic SIP Strategy was not trained on the AMZN price history, and used the weights from its training under the SPX price history from 2000 to 2020. If it were to be trained under the AMZN stock price history instead, its performance most likely would increase and a similar gap would form between it and the Fixed SIP Strategy.

Golden Cross and Momentum both severely underperformed Fixed SIP on AMZN, finishing at $902,881 and $1,294,960 respectively against Fixed SIP's $4,026,430. Every period spent in cash on an asset compounding at roughly 19% annually carries an enormous opportunity cost, and with only 20 trades over 20 years the Golden Cross Strategy missed multiple multi-year bull runs in their entirety.

<u>Does Dynamic SIP Beat Fixed SIP? Under What Conditions?</u>

Based on the backtesting results, Dynamic SIP outperforms Fixed SIP on both assets tested, but the magnitude depends heavily on the characteristics of the underlying asset. An important caveat is that the eight parameters of the Dynamic SIP Strategy were optimized exclusively on SPX data from 2000 to 2020 and were not retrained on AMZN. This means the weights were tuned for index-like behavior and are not necessarily well-suited for a high-growth individual stock like AMZN.

On a diversified index like SPX, the improvement is compelling, showing higher return, lower drawdown, and better risk-adjusted performance. The death cross component avoids the worst of multi-year bear markets while monthly contributions accumulate as uninvested cash, and when the golden cross fires on recovery, this accumulated cash is deployed at discounted prices, amplifying the recovery gain. The tiered dip-buying logic further boosts performance by scaling up investment during pullbacks, which are common and recoverable on a broad index. This combination of market timing and dip sensitivity is well matched to the boom-bust cycle that SPX experienced between 2000 and 2020.

On AMZN, Dynamic SIP still wins on final value but only marginally and with no meaningful drawdown reduction. An asset compounding at roughly 19% annually punishes any strategy that sits in cash, and Fixed SIP's mechanical indifference to price direction becomes an advantage since it never exits the position and never misses a recovery. The mild dip threshold of 2.75% also becomes less meaningful on a stock as volatile as AMZN, where a 2.75% daily swing is relatively common and does not carry the same signal value as it would on a slower-moving index. Additionally, since the parameters were not optimized for AMZN, it is likely that if the parameter sweep were re-run on AMZN price history, a different set of weights would be found that would produce a larger gap over Fixed SIP similar to what was seen on SPX.

In general, Dynamic SIP is expected to outperform most strongly on mean-reverting assets such as broad indices, during periods containing prolonged bear markets that last long enough for the moving average crossover to fire, and on assets where dips are deep and

recoverable within the lookback window. Its advantage weakens on sustained parabolic growth assets, during very short crashes that resolve before the death cross triggers, and on highly volatile assets where the mild dip threshold fires frequently on noise, diluting the tiered buying logic. Overall the results suggest that the Dynamic SIP Strategy is a meaningful improvement over Fixed SIP when the asset and market conditions are a good match for its design, but parameter retraining is important when applying it to a new asset class.

[AI Usage — Section 4: Results and Analysis]

Prompt: "ok expand sweep to have it vary a lot more" AI Output: Expanded parameter sweep from 3 variables to 6, increasing combinations from ~1683 to ~13500. Modification: Further expanded to 8 parameters including rally suppression variables for the final sweep.

Prompt: "where does it show the best / is this the best strategy?" AI Output: Explained that BST inorder prints worst to best so the last printed entry is the highest final value. Modification: Used this to correctly interpret sweep results and identify the optimal parameter set reported in the results section.

Prompt: "explain how this works / explain all the parameters / does dynamic sip beat fixed sip?" AI Output: Provided detailed explanations of strategy logic and parameter interactions for each of the eight parameters. Modification: Used these explanations as the basis for the written analysis in this section, rewritten in my own words.

Conclusions

This project successfully built a fully functional stock market simulator in C++ that applied every major data structure covered in ESE 224 to a real world financial problem. The motivating question of whether a dynamic systematic investment plan could outperform a fixed one was answered clearly. Yes, at least on SPX over the 2000 to 2020 period, and by a meaningful margin in both return and drawdown. The Dynamic SIP Strategy produced a final value of $401,601.47 against Fixed SIP's $276,886.13, while also reducing maximum drawdown

by over 16 percentage points. This demonstrated that a parameter optimized hybrid strategy combining moving average crossovers with tiered dip buying can add genuine value over a naive monthly investment approach. However, there is a massive problem with approaching the stock market with this strategy as the Dynamic SIP Strategy was optimized to provide the highest returns only for the 2000 to 2020 period. If this strategy were to be implemented using today's stock market, it would return significantly less. This characterizes the inherent nature of the stock market as almost all niches within the market are filled by larger investment firms seeking return: A trading algorithm on this scale would simply not suffice.

From a data structures perspective, the project reinforced how important choosing the right structure for the right problem is. The doubly linked list was a natural fit for chronological price data, the circular queue made rolling average computation straightforward, the BST allowed sorted ranking of strategy results, and the stack enabled clean undo functionality. Trying to use any of these structures for a different role would have introduced unnecessary complexity, and the project made those tradeoffs concrete in a way that coursework problems alone do not.

There are several notable limitations to this work. The simulation does not account for transaction costs, taxes, or bid ask spread, all of which would reduce real world returns especially for the Dynamic SIP Strategy which executed 461 trades compared to Fixed SIP's 240. The momentum and golden cross strategies also used fixed parameters rather than being swept, so their results may not represent their true potential.

If this project were to be extended, the most impactful improvement would be adding a proper train test split for the parameter sweep so that the optimized weights could be validated on out of sample data. It would also be interesting to run the parameter sweep separately on AMZN and NVDA to see how much the optimal weights differ across asset types, which would give a better picture of how asset specific the Dynamic SIP design really is. Finally, adding transaction cost modeling would make the simulation more realistic and would likely change the relative rankings of the strategies, since higher trade count strategies like Dynamic SIP would be penalized more than Fixed SIP in a real brokerage environment.